

Relazione di Progettazione: Architettura Modulare e Design Pattern nel Sistema ModShop

Un'analisi tecnica approfondita dell'architettura software di ModShop, piattaforma e-commerce progettata per rispondere alle sfide di un mercato dinamico attraverso l'applicazione sistematica dei Design Pattern GoF. Il documento illustra come manutenibilità, estensibilità e separazione delle responsabilità siano stati tradotti in componenti software disaccoppiati e scalabili.

ARCHITETTURA SOFTWARE

DESIGN PATTERN GOF

E-COMMERCE

Visione Architettuale e Obiettivi del Sistema

Il progetto ModShop è stato concepito per rispondere alle sfide di un mercato e-commerce sempre più dinamico, dove la capacità di adattare l'offerta e personalizzare l'esperienza d'acquisto rappresenta un asset strategico imprescindibile. In qualità di Software Architect, l'obiettivo primario è stato lo sviluppo di un'architettura che non si limitasse a gestire transazioni, ma che fungesse da framework estensibile e resiliente.

Il sistema è fondato su tre pilastri fondamentali che ne definiscono l'identità tecnica e la solidità strutturale. Questi obiettivi sono stati perseguiti per gestire efficacemente prodotti fisici che richiedono configurazioni granulari e logiche di pricing variabili, trasformando requisiti di business complessi in componenti software disaccoppiati attraverso l'applicazione dei Design Pattern GoF.

Manutenibilità

Capacità di intervenire su un modulo senza rischio di regressioni sugli altri componenti del sistema.

Estensibilità

Aggiunta di nuove funzionalità per estensione, non per modifica del codice esistente.

Separation of Concerns

Rigorosa separazione delle responsabilità tra i moduli per garantire coesione e basso accoppiamento.

Singleton Pattern: Centralizzazione dello Stato e Configurazione Globale

In un ecosistema e-commerce, l'integrità dei dati finanziari è prioritaria. La gestione di parametri critici come l'IVA, la valuta e gli sconti base richiede una **"Single Source of Truth"** per evitare inconsistenze tra i diversi moduli di calcolo. Il sistema risolve questa necessità tramite la classe `AppContext`, implementata come Singleton.

La classe è dichiarata `sealed` per impedirne l'ereditarietà e presenta un costruttore `private`, garantendo che l'istanza sia unica e controllata. L'accesso avviene esclusivamente tramite la proprietà statica `getInstance`, che utilizza un controllo di esistenza (`if (_instance == null)`) per l'inizializzazione lazy. Dal punto di vista architettonico, l'uso del Singleton assicura che ogni componente del sistema interroghi la medesima configurazione, facilitando aggiornamenti massivi delle politiche fiscali senza rifattorizzare i moduli periferici.

Proprietà Gestite da AppContext

- **Valuta:** Accetta esclusivamente codici di 3 caratteri (es. "EUR", "USD"), validando `value.Length == 3`.
- **IVA:** Logica di fallback robusta; se il valore non è superiore a zero, imposta automaticamente il default a **23.0d**.
- **ScontoBase:** Percentuale di sconto globale, soggetta a validazione per prevenire valori negativi.

Definizione dei Prodotti Core e Interfaccia IProduct

La struttura del catalogo ModShop poggia sull'interfaccia `IProduct`, che funge da contratto fondamentale per tutti i beni commerciabili. Questa astrazione definisce i metodi `GetName()` e `GetPrice()`, consentendo al sistema di trattare in modo polimorfico magliette, tazze o accessori durante le fasi di calcolo del carrello. L'astrazione tramite interfaccia permette al motore di calcolo di iterare su una lista di oggetti `IProduct` e calcolare il totale senza conoscere le classi concrete (`TShirt`, `Mug`, `Skin`).

I prodotti base attualmente censiti nel sistema costituiscono il nucleo del catalogo, ciascuno con un prezzo base definito e un codice identificativo univoco. Tuttavia, per superare la staticità dei prodotti base e permettere la personalizzazione richiesta dal business, il sistema evolve verso un modello dinamico basato sul Decorator Pattern.

Codice Prodotto	Descrizione Prodotto	Prezzo Base
TSHIRT	Maglietta in cotone	€ 20.00
MUG	Tazza in ceramica	€ 10.00
SKIN	Pellicola protettiva	€ 5.00

Decorator Pattern: Estensibilità Dinamica e Personalizzazione

Per implementare opzioni aggiuntive (come stampe o incisioni) rispettando il principio **Open/Closed**, è stato adottato il pattern Decorator. Invece di creare una sottoclasse per ogni possibile combinazione di prodotto e opzione, il sistema utilizza la composizione rispetto all'ereditarietà. La classe astratta `ProductDecorator` implementa `IProduct` e contiene un riferimento protetto `_product`, permettendo di "incapsulare" un prodotto esistente aggiungendo comportamenti o costi in modo incrementale.

L'impatto tecnico è significativo: il sistema può generare una `Mug` con `Incisione` e `ConfezioneRegalo` semplicemente stratificando gli oggetti. Questo approccio evita l'esplosione combinatoria delle classi e permette di aggiungere nuove opzioni di personalizzazione creando un nuovo decoratore, senza toccare il codice dei prodotti core.

	StampaFronte Costo aggiuntivo: +€ 3.50
	StampaRetro Costo aggiuntivo: +€ 3.20
	ConfezioneRegalo Costo aggiuntivo: +€ 1.50
	Incisione Costo aggiuntivo: +€ 4.00
	EstensioneGaranzia Costo aggiuntivo: +€ 5.50

Strategy Pattern: Flessibilità nelle Logiche di Pricing

La determinazione del prezzo finale è una logica suscettibile di frequenti cambiamenti — campagne marketing, vendite all'ingrosso, normative fiscali. Lo **Strategy Pattern** permette di isolare l'algoritmo di calcolo dalla struttura del prodotto. Le strategie implementate ereditano dall'interfaccia `IStrategia` e vengono istanziate passando i parametri correnti dal Singleton `AppContext`. L'integrazione è gestita tramite il metodo `SetStrategia`, che consente al sistema di cambiare comportamento dinamico a runtime — ad esempio, attivando la `PromoPricing` durante un evento stagionale senza modificare una singola riga di codice nel catalogo prodotti.



Questo disaccoppiamento tra logica di calcolo e struttura del prodotto rappresenta uno dei contributi architetturali più rilevanti del sistema, garantendo una flessibilità operativa senza precedenti nella gestione delle politiche commerciali.

Observer Pattern: Disaccoppiamento della Notifica e Logging

Un'architettura enterprise deve essere reattiva. Nel sistema ModShop, ogni variazione di stato di un prodotto — creazione, modifica, rimozione — deve essere comunicata a sottosistemi indipendenti come il logging di sistema o l'interfaccia utente.

L'**Observer Pattern** gestisce questa reattività tramite l'interfaccia `IProductObserver`, che definisce il metodo `void Aggiorna(IProduct product, string messaggio)`.

Ogni classe prodotto (`Mug`, `Skin`, `TShirt`) mantiene una lista interna di osservatori (`IstObs`). Quando viene invocato un evento, il prodotto notifica tutti i soggetti registrati inviando un messaggio contestuale. Questo disaccoppiamento garantisce che il prodotto non debba conoscere i dettagli implementativi dei sistemi di log o delle interfacce, migliorando drasticamente la modularità e la facilità di test.



Log

Registra i dettagli tecnici dell'operazione per il monitoraggio e il debugging del sistema in produzione.



UI

Aggiorna gli elementi visuali per l'utente finale, garantendo un'esperienza d'acquisto sempre sincronizzata con lo stato reale del catalogo.



Mock

Fornisce un endpoint dedicato per i test di integrazione, permettendo la simulazione degli eventi senza impatto sui sistemi reali.

Analisi dei Vantaggi e Conclusioni Tecniche

L'adozione sinergica di questi quattro Design Pattern ha trasformato ModShop in una piattaforma professionale, pronta per la scalabilità orizzontale e verticale. L'architettura non è più una sequenza di istruzioni, ma un insieme di componenti intelligenti che collaborano tramite interfacce definite. La solidità dell'architettura ModShop fornisce una base tecnica d'eccellenza, capace di supportare le future evoluzioni del business con un debito tecnico ridotto al minimo e un'altissima manutenibilità del codice.

Scalabilità

L'aggiunta di nuovi prodotti o decoratori avviene per estensione, non per modifica del codice esistente, eliminando il rischio di regressioni.

Flessibilità

Il sistema può cambiare logiche di pricing a runtime tramite l'iniezione di strategie nel contesto globale, senza interventi sul catalogo prodotti.

Robustezza

La centralizzazione tramite Singleton e la validazione dei dati — come il fallback dell'IVA al **23%** — prevengono errori a catena nel calcolo finanziario.

Manutenibilità

La netta separazione delle responsabilità permette di intervenire su un modulo (es. le notifiche) senza rischio di regressioni sul calcolo dei prezzi o sulle personalizzazioni.

- ❏ La solidità dell'architettura ModShop fornisce una base tecnica d'eccellenza, capace di supportare le future evoluzioni del business con un debito tecnico ridotto al minimo.